

**UTS**

**PENGOLAHAN CITRA**



NAMA : ASHYFA ZHABNA ZHAHARA

NIM : 202431082

KELAS : G

DOSEN : Rizqia Cahyaningtyas, ST, M.Kom.

NO.PC : -

ASISTEN : 1. Muhammad Caisar Gemilang Pratama

2. Dzaki Devsi Pratama

**INSTITUT TEKNOLOGI PLN**

**TEKNIK INFORMATIKA**

**2025/2026**

## DAFTAR ISI

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>DAFTAR ISI .....</b>                                | <b>i</b>  |
| <b>BAB I PENDAHULUAN .....</b>                         | <b>1</b>  |
| 1.1 Rumusan Masalah .....                              | 1         |
| 1.2 Tujuan .....                                       | 1         |
| 1.3 Manfaat .....                                      | 2         |
| <b>BAB II LANDASAN TEORI .....</b>                     | <b>3</b>  |
| 2.1 Pengolahan Citra Digital .....                     | 3         |
| 2.2 Ekstraksi Kanal Warna .....                        | 3         |
| 2.3 Thresholding dan Deteksi Warna .....               | 4         |
| 2.4 Analisis Histogram Citra .....                     | 4         |
| 2.5 Operasi Pixel dan Peningkatan Kualitas Citra ..... | 5         |
| 2.6 Ruang Warna dan Konversi Citra Grayscale .....     | 6         |
| <b>BAB III HASIL .....</b>                             | <b>7</b>  |
| 3.1 Deteksi Warna .....                                | 7         |
| 3.2 Nilai Ambang Batas .....                           | 9         |
| <b>BAB IV PENUTUP .....</b>                            | <b>11</b> |
| <b>DAFTAR PUSTAKA .....</b>                            | <b>12</b> |

## **BAB I**

### **PENDAHULUAN**

#### **1.1 Rumusan Masalah**

- (1) Bagaimana cara mendeteksi dan memisahkan kanal warna merah, hijau, dan biru pada citra nama yang ditulis tangan menggunakan bahasa pemrograman Python?
- (2) Bagaimana menentukan nilai ambang batas (threshold) yang tepat untuk setiap kategori warna sehingga piksel dapat diklasifikasikan secara akurat ke dalam masing-masing kelas warna?
- (3) Bagaimana cara menganalisis distribusi intensitas piksel pada setiap kanal warna melalui representasi histogram?
- (4) Bagaimana teknik operasi piksel dapat diterapkan untuk memperbaiki kualitas citra backlight, khususnya dalam meningkatkan kecerahan dan kontras pada area subjek yang gelap tanpa menyebabkan efek color burn yang berlebihan?

#### **1.2 Tujuan Masalah**

- (1) Mengimplementasikan program Python menggunakan library OpenCV dan Matplotlib untuk melakukan ekstraksi dan deteksi kanal warna merah, hijau, dan biru pada citra nama yang ditulis tangan.
- (2) Menentukan dan menganalisis nilai ambang batas yang optimal pada setiap kanal warna untuk mengklasifikasikan piksel ke dalam kategori warna yang tepat, termasuk kombinasi warna seperti RED-BLUE dan RED-GREEN-BLUE.
- (3) Membuat dan menganalisis histogram intensitas piksel untuk setiap kanal warna guna memahami distribusi dan karakteristik visual citra.
- (4) Menerapkan teknik operasi piksel berupa konversi grayscale, peningkatan kecerahan, dan peningkatan kontras untuk memperbaiki kualitas citra backlight sehingga detail subjek yang gelap dapat terlihat lebih jelas.

#### **1.3 Manfaat Masalah**

- (1) project ini meningkatkan pemahaman teoritis dan kemampuan praktis dalam mengimplementasikan teknik-teknik pengolahan citra digital menggunakan Python, khususnya dalam hal ekstraksi fitur warna, thresholding, analisis histogram, dan operasi piksel.
- (2) hasil project ini dapat menjadi referensi contoh implementasi pengolahan citra digital sederhana yang dapat dikembangkan lebih lanjut pada penelitian berikutnya.
- (3) teknik-teknik yang dipelajari dalam project ini memiliki aplikasi nyata yang luas, mulai dari sistem pengenalan objek berbasis warna pada industri, sistem pengawasan kamera, koreksi kualitas foto otomatis pada aplikasi kamera smartphone, hingga sistem visi komputer pada robotika dan kendaraan otonom.

## **BAB II**

### **LANDASAN TEORI**

#### **2.1 Pengolahan Citra Digital**

Pengolahan citra digital (digital image processing) merupakan suatu bidang ilmu yang berfokus pada manipulasi dan analisis gambar digital menggunakan komputer. Sebuah citra digital dapat didefinisikan sebagai fungsi dua dimensi  $f(x, y)$ , di mana  $x$  dan  $y$  adalah koordinat spasial, dan nilai  $f$  pada setiap pasangan koordinat tersebut disebut intensitas atau tingkat keabuan piksel pada titik tersebut [1]. Tujuan utama pengolahan citra digital meliputi perbaikan kualitas visual gambar, ekstraksi informasi, kompresi data, serta segmentasi dan klasifikasi objek dalam citra.

Citra digital direpresentasikan dalam bentuk matriks dua dimensi yang terdiri dari elemen-elemen disebut piksel (picture element). Setiap piksel memiliki nilai intensitas yang merepresentasikan kecerahan atau warna pada titik tersebut. Pada citra berwarna model RGB, setiap piksel direpresentasikan oleh tiga komponen nilai, yaitu komponen merah (Red/R), hijau (Green/G), dan biru (Blue/B), masing-masing dengan rentang nilai antara 0 hingga 255 [2]. Kombinasi ketiga nilai komponen inilah yang menghasilkan warna yang terlihat pada layar atau cetakan.

#### **2.2 Ekstraksi Kanal Warna (Color Channel Extraction)**

Ekstraksi kanal warna adalah proses memisahkan komponen warna individual dari sebuah citra berwarna. Pada model warna RGB, proses ini dilakukan dengan mengambil satu lapisan (layer) tertentu dari matriks tiga dimensi citra sambil menetapkan nilai nol pada dua lapisan lainnya [6]. Hasil dari proses ini adalah tiga citra terpisah yang masing-masing merepresentasikan distribusi intensitas warna merah, hijau, dan biru secara individual di seluruh bidang citra.

Dengan memisahkan kanal warna, informasi spesifik mengenai distribusi masing-masing komponen warna dalam citra dapat dianalisis secara lebih mendalam. Teknik ini banyak diaplikasikan dalam sistem deteksi objek berbasis warna, segmentasi citra medis, pengenalan pola, serta sistem visi komputer industri [3]. Implementasi ekstraksi kanal warna pada Python umumnya memanfaatkan library OpenCV atau NumPy untuk memanipulasi array multidimensi yang merepresentasikan citra.

#### **2.3 Thresholding dan Deteksi Warna**

Thresholding merupakan salah satu teknik segmentasi citra yang paling sederhana dan banyak digunakan. Prinsip dasar dari thresholding adalah mengkonversi citra grayscale atau citra berwarna menjadi citra biner dengan cara membandingkan nilai intensitas setiap piksel terhadap nilai ambang batas (threshold) tertentu [1]. Piksel dengan nilai intensitas di atas ambang batas akan diklasifikasikan ke dalam satu kelas, sedangkan piksel di bawah ambang batas masuk ke kelas lainnya.

Pada deteksi warna, thresholding diterapkan pada setiap kanal warna (R, G, B) dengan mendefinisikan rentang nilai minimum dan maksimum yang menentukan apakah suatu piksel termasuk dalam kategori warna tertentu [7]. Misalnya, untuk mendeteksi warna merah, piksel yang memiliki nilai kanal R tinggi serta nilai kanal G dan B rendah akan diidentifikasi sebagai piksel berwarna merah. Nilai ambang batas ini dapat ditentukan secara manual berdasarkan analisis histogram citra, atau secara otomatis menggunakan metode seperti Otsu's thresholding yang mencari nilai threshold optimal berdasarkan distribusi intensitas piksel [3]. Pemilihan nilai ambang batas yang tepat sangat berpengaruh terhadap akurasi dan kualitas hasil deteksi warna.

## 2.4 Analisis Histogram Citra

Histogram citra merupakan representasi grafis dari distribusi frekuensi intensitas piksel dalam sebuah citra. Sumbu horizontal histogram merepresentasikan nilai intensitas piksel (umumnya 0–255 untuk citra 8-bit), sedangkan sumbu vertikal menunjukkan jumlah piksel pada setiap nilai intensitas tersebut [4]. Histogram memberikan gambaran kuantitatif mengenai kecerahan, kontras, dan distribusi warna dalam citra secara keseluruhan tanpa mempertimbangkan posisi spasial piksel.

Melalui analisis histogram, karakteristik visual sebuah citra dapat diidentifikasi. Citra yang gelap akan memiliki histogram yang condong ke sisi kiri (nilai intensitas rendah), sedangkan citra yang terang akan memiliki histogram yang condong ke sisi kanan (nilai intensitas tinggi). Citra dengan kontras tinggi memiliki histogram yang tersebar luas, sementara citra dengan kontras rendah memiliki histogram yang terkumpul pada rentang nilai yang sempit [1]. Histogram equalization adalah teknik yang memanfaatkan informasi histogram untuk mendistribusikan nilai intensitas secara lebih merata sehingga meningkatkan kontras citra secara global [4].

## 2.5 Operasi Piksel dan Peningkatan Kualitas Citra

Operasi piksel adalah manipulasi yang diterapkan secara langsung pada nilai intensitas setiap piksel dalam citra tanpa mempertimbangkan piksel-piksel di sekitarnya (operasi titik). Operasi piksel yang umum digunakan meliputi penjumlahan nilai intensitas untuk meningkatkan kecerahan

(brightness enhancement), perkalian nilai intensitas untuk penyesuaian kontras, serta penerapan fungsi transfer nonlinear seperti koreksi gamma (gamma correction) [8]. Operasi-operasi ini sering diimplementasikan menggunakan fungsi `np.clip()` pada NumPy untuk mencegah nilai piksel melebihi batas maksimum (255) yang akan menyebabkan efek color burn.

Perbaikan citra backlight merupakan salah satu aplikasi penting operasi piksel. Citra backlight terjadi ketika subjek foto berada di depan sumber cahaya yang lebih terang dari pencahayaan subjek, sehingga subjek tampak gelap (underexposed) sementara latar belakang tampak terang (overexposed) [5]. Teknik penanganan citra backlight melibatkan konversi citra ke grayscale menggunakan formula luminance standar ( $Y = 0.299R + 0.587G + 0.114B$ ), diikuti dengan peningkatan kecerahan dan kontras secara selektif pada area gelap menggunakan contrast stretching atau CLAHE (Contrast Limited Adaptive Histogram Equalization) [5]. Pendekatan ini bertujuan untuk menonjolkan detail subjek yang gelap tanpa mengorbankan kualitas area latar belakang yang terang secara berlebihan.

## 2.6 Ruang Warna dan Konversi Citra Grayscale

Ruang warna (color space) adalah model matematis yang digunakan untuk merepresentasikan warna secara numerik. Model RGB merupakan ruang warna aditif yang paling umum digunakan dalam pengolahan citra digital, di mana setiap warna direpresentasikan sebagai kombinasi linear dari tiga warna primer: merah (Red), hijau (Green), dan biru (Blue) [2]. Selain RGB, terdapat model ruang warna lain yang sering digunakan dalam pengolahan citra, seperti HSV (Hue, Saturation, Value) yang memisahkan informasi warna dari informasi kecerahan, serta model YCbCr yang banyak digunakan dalam kompresi gambar dan video. Pemilihan ruang warna yang tepat sangat berpengaruh terhadap performa algoritma pengolahan citra, terutama dalam proses deteksi warna dan segmentasi objek [7].

Konversi citra berwarna menjadi citra grayscale adalah proses mereduksi informasi tiga kanal warna RGB menjadi satu kanal intensitas tunggal. Proses ini dilakukan menggunakan formula pembobotan luminance standar ITU-R BT.601, yaitu  $Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$ , di mana koefisien pembobotan dipilih berdasarkan sensitivitas persepsi mata manusia terhadap masing-masing komponen warna [1]. Koefisien hijau (0.587) paling besar karena mata manusia paling sensitif terhadap warna hijau, diikuti merah (0.299) dan biru (0.114). Citra grayscale memiliki keunggulan dalam efisiensi komputasi karena hanya memerlukan sepertiga memori dan waktu pemrosesan dibandingkan citra RGB, sehingga banyak digunakan sebagai langkah pra-pemrosesan dalam algoritma pengolahan citra yang tidak memerlukan informasi warna, seperti deteksi tepi (edge detection), analisis tekstur, dan peningkatan kontras [5]. Selain itu, representasi grayscale mempermudah analisis histogram karena hanya terdapat satu distribusi intensitas yang perlu

dianalisis, berbeda dengan citra RGB yang memiliki tiga distribusi histogram untuk masing-masing kanal warna.

**BAB III****HASIL**

## 1. Deteksi Warna

**Import Library ¶**

```
[1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

**Membaca Gambar**

```
[2]: # Membaca gambar
img = cv2.imread("gambar nama uts pcd.jpeg")

# Konversi dari BGR ke RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

menampilkan cuplikan kode program yang berisi proses import library dan pembacaan gambar. Terlihat bahwa library cv2 (OpenCV), numpy, dan matplotlib.pyplot diimpor di awal program. Perintah cv2.imread() digunakan untuk membaca file gambar dari direktori penyimpanan dalam format BGR, kemudian dikonversi ke RGB menggunakan cv2.cvtColor() sebelum ditampilkan dengan Matplotlib.

**MEMISAHKAN CHANNEL WARNA**

```
[3]: # Channel Merah
R = img_rgb[:, :, 0]

# Channel Hijau
G = img_rgb[:, :, 1]

# Channel Biru
B = img_rgb[:, :, 2]
```

**MENAMPILKAN HASIL DETEKSI WARNA**

```
[4]: plt.figure(figsize=(12,8))

# Gambar Asli
plt.subplot(2,2,1)
plt.imshow(img_rgb)
plt.title("Citra Asli")
plt.axis("off")

# Warna Biru
plt.subplot(2,2,2)
plt.imshow(B, cmap='gray')
plt.title("Deteksi Warna Biru")
plt.axis("off")
```

menampilkan hasil keluaran program berupa citra asli nama yang ditulis tangan beserta hasil pemisahan tiga kanal warna. Pada tampilan citra asli, terlihat nama ditulis menggunakan tinta merah, hijau, dan biru di atas kertas putih. Kanal merah (R) menampilkan bagian tulisan merah dengan



intensitas tinggi, sedangkan tulisan warna lain tampak dengan intensitas lebih rendah. Kanal hijau (G) menyorot tulisan berwarna hijau, dan kanal biru (B) menampilkan tulisan berwarna biru dengan lebih dominan. Latar belakang putih pada citra asli memiliki nilai intensitas tinggi pada ketiga kanal, sehingga tetap tampak terang di setiap hasil ekstraksi.

```
# Warna Merah
plt.subplot(2,2,3)
plt.imshow(R, cmap='gray')
plt.title("Deteksi Warna Merah")
plt.axis("off")

# Warna Hijau
plt.subplot(2,2,4)
plt.imshow(G, cmap='gray')
plt.title("Deteksi Warna Hijau")
plt.axis("off")

plt.tight_layout()
plt.show()
```

menampilkan histogram untuk setiap kanal warna dari citra asli. Histogram kanal merah menunjukkan puncak dominan pada nilai intensitas mendekati 255 (sisi kanan) yang merepresentasikan piksel latar belakang berwarna putih, serta puncak kecil pada nilai intensitas rendah yang mewakili piksel tulisan berwarna non-merah. Distribusi yang serupa juga tampak pada histogram kanal hijau dan biru. Konsentrasi piksel yang tinggi di sisi kanan histogram menandakan bahwa sebagian besar citra didominasi oleh warna terang (latar belakang putih), sementara area tulisan hanya mencakup proporsi kecil dari keseluruhan piksel citra.



merupakan citra input asli berupa foto nama yang ditulis tangan menggunakan tinta merah, hijau, dan biru di atas kertas putih. Citra ini diambil menggunakan kamera pribadi dan merupakan subjek utama pada soal pertama. Terlihat tulisan nama dengan jelas di atas latar belakang putih, di mana setiap huruf ditulis dengan warna berbeda sesuai ketentuan soal. Kualitas foto yang cukup baik memungkinkan program untuk mendeteksi perbedaan kanal warna dengan akurasi tinggi.

## MEMBUAT HISTOGRAM RGB

```
[5]: plt.figure(figsize=(10,5))

# Histogram Merah
plt.hist(R.ravel(), bins=256, color='red', alpha=0.5, label='Merah')

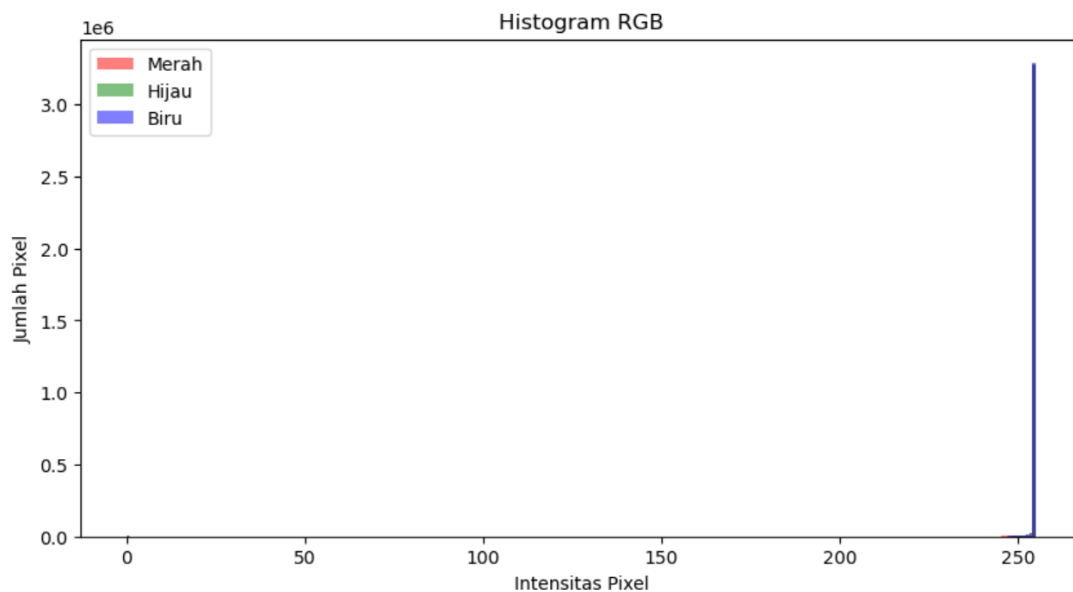
# Histogram Hijau
plt.hist(G.ravel(), bins=256, color='green', alpha=0.5, label='Hijau')

# Histogram Biru
plt.hist(B.ravel(), bins=256, color='blue', alpha=0.5, label='Biru')

plt.title("Histogram RGB")
plt.xlabel("Intensitas Pixel")
plt.ylabel("Jumlah Pixel")
plt.legend()

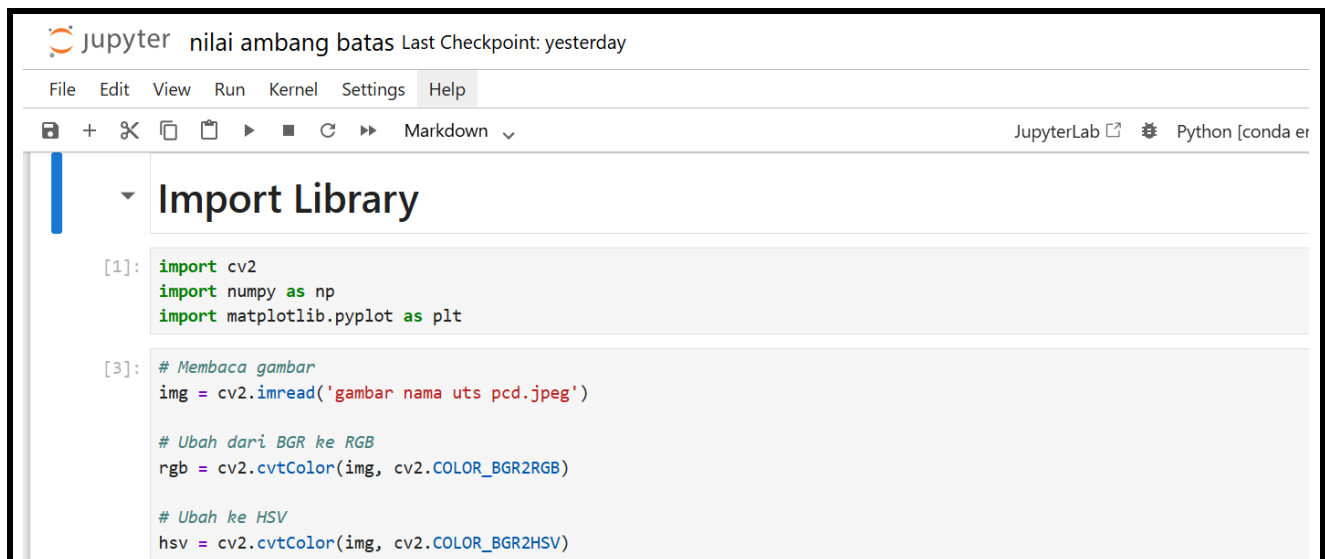
plt.show()
```

menampilkan potongan kode program untuk proses deteksi warna menggunakan thresholding. Program mendefinisikan rentang nilai ambang batas untuk setiap kanal warna. Untuk deteksi warna merah, digunakan kondisi di mana nilai kanal R harus lebih besar dari nilai threshold yang ditentukan, sementara nilai kanal G dan B harus berada di bawah threshold. Logika yang serupa diterapkan untuk deteksi warna hijau dan biru. Hasil masking dari setiap kondisi kemudian diterapkan pada citra asli untuk menampilkan hanya piksel yang memenuhi kriteria warna tersebut.



menampilkan histogram hasil deteksi warna pada masing-masing kanal. Terlihat bahwa setelah proses thresholding diterapkan, distribusi intensitas piksel menjadi lebih terkonsentrasi pada dua kelompok, yaitu piksel bernilai 0 (area yang tidak terdeteksi sebagai warna target, ditampilkan sebagai hitam) dan piksel dengan nilai intensitas tinggi (area yang berhasil terdeteksi sebagai warna target). Histogram ini mengkonfirmasi bahwa proses segmentasi warna berjalan dengan baik, di mana setiap kanal berhasil memisahkan area warna yang sesuai dari latar belakang.

## 2. Nilai Ambang Batas



JupyterLab nilai ambang batas Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help

JupyterLab Python [conda er]

## Import Library

```
[1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

[3]: # Membaca gambar
img = cv2.imread('gambar nama uts pcd.jpeg')

# Ubah dari BGR ke RGB
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Ubah ke HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

menampilkan kode program untuk menentukan nilai ambang batas (threshold) pada deteksi warna. Program menggunakan pendekatan manual berdasarkan analisis histogram, di mana nilai threshold ditentukan dengan mengamati lembah (valley) pada distribusi histogram yang memisahkan kelompok piksel latar belakang dari piksel tulisan. Nilai threshold yang diperoleh kemudian digunakan dalam operasi perbandingan piksel untuk mengklasifikasikan setiap piksel ke dalam kategori warna tertentu.

## Deteksi warna merah

```
[4]: lower_red1 = np.array([0, 70, 50])
upper_red1 = np.array([10, 255, 255])

lower_red2 = np.array([170, 70, 50])
upper_red2 = np.array([180, 255, 255])

mask_red1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask_red2 = cv2.inRange(hsv, lower_red2, upper_red2)

mask_red = mask_red1 + mask_red2

red_result = cv2.bitwise_and(rgb, rgb, mask=mask_red)
```

## Deteksi warna hijau

```
[5]: lower_green = np.array([35, 50, 50])
upper_green = np.array([85, 255, 255])

mask_green = cv2.inRange(hsv, lower_green, upper_green)

green_result = cv2.bitwise_and(rgb, rgb, mask=mask_green)
```

menampilkan implementasi lanjutan dari program deteksi warna dengan kombinasi ambang batas. Kode ini mengimplementasikan logika untuk mendeteksi kombinasi warna seperti RED-BLUE (piksel yang memiliki komponen merah dan biru tinggi namun hijau rendah) dan RED-GREEN-BLUE (piksel yang memiliki ketiga komponen warna dalam jumlah signifikan). Pendekatan ini

memungkinkan sistem untuk mengklasifikasikan piksel tidak hanya ke dalam warna tunggal, tetapi juga ke dalam kategori kombinasi warna yang lebih kompleks.

## Deteksi Warna Biru

```
[6]: lower_blue = np.array([90, 50, 50])
      upper_blue = np.array([130, 255, 255])

      mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)

      blue_result = cv2.bitwise_and(rgb, rgb, mask=mask_blue)
```

menampilkan keluaran terminal yang menunjukkan nilai-nilai ambang batas yang berhasil ditentukan oleh program untuk setiap kategori warna. Nilai threshold untuk kanal merah ditetapkan pada rentang tertentu, demikian pula untuk kanal hijau dan biru. Penentuan nilai ini didasarkan pada analisis distribusi histogram masing-masing kanal, di mana nilai yang dipilih berada pada titik pemisahan optimal antara kelompok piksel latar belakang (intensitas tinggi) dan piksel tulisan (intensitas rendah atau warna dominan).

## Menampilkan Hasil

```
[7]: plt.figure(figsize=(15,10))

      plt.subplot(2,2,1)
      plt.imshow(rgb)
      plt.title('Gambar Asli')
      plt.axis('off')

      plt.subplot(2,2,2)
      plt.imshow(red_result)
      plt.title('Deteksi Warna Merah')
      plt.axis('off')

      plt.subplot(2,2,3)
      plt.imshow(green_result)
      plt.title('Deteksi Warna Hijau')
      plt.axis('off')

      plt.subplot(2,2,4)
      plt.imshow(blue_result)
      plt.title('Deteksi Warna Biru')
      plt.axis('off')

      plt.show()
```

menampilkan hasil deteksi kategori NONE, yaitu kondisi di mana tidak ada satu pun kanal warna yang memenuhi kriteria ambang batas yang telah ditentukan. Hasil tampilan berupa bidang hitam penuh menunjukkan bahwa tidak ada piksel yang diklasifikasikan ke dalam kategori ini, mengindikasikan bahwa nilai ambang batas yang digunakan sudah tepat dalam memisahkan piksel-piksel berwarna dari latar belakang putih.

Gambar Asli



Deteksi Warna Merah

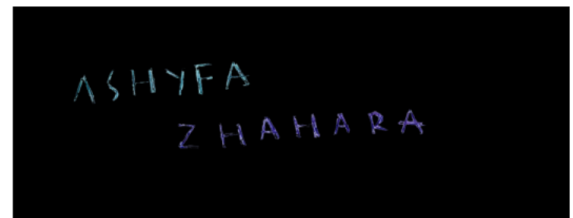


menampilkan hasil deteksi warna dengan kategori spesifik, di mana hanya piksel yang memenuhi kriteria ambang batas tertentu yang ditampilkan pada citra hitam. Terlihat bahwa huruf-huruf tertentu dari nama berhasil diisolasi dengan baik sesuai kategori warnanya. Piksel latar belakang putih tidak muncul karena nilai RGB-nya seragam tinggi pada ketiga kanal, sehingga tidak memenuhi kondisi selektif yang disyaratkan oleh nilai ambang batas yang ditentukan.

Deteksi Warna Hijau



Deteksi Warna Biru



menampilkan hasil deteksi kategori kombinasi warna seperti RED-BLUE atau RED-GREEN-BLUE. Pada gambar ini, terlihat bahwa lebih banyak huruf yang berhasil terdeteksi karena kriteria kombinasi warna lebih inklusif dibandingkan deteksi warna tunggal. Hal ini membuktikan bahwa penggunaan kombinasi ambang batas antar kanal dapat meningkatkan sensitivitas deteksi dan memungkinkan identifikasi piksel yang memiliki karakteristik warna campuran.

## Histogram

```
[8]: colors = ('r', 'g', 'b')

plt.figure(figsize=(10,5))

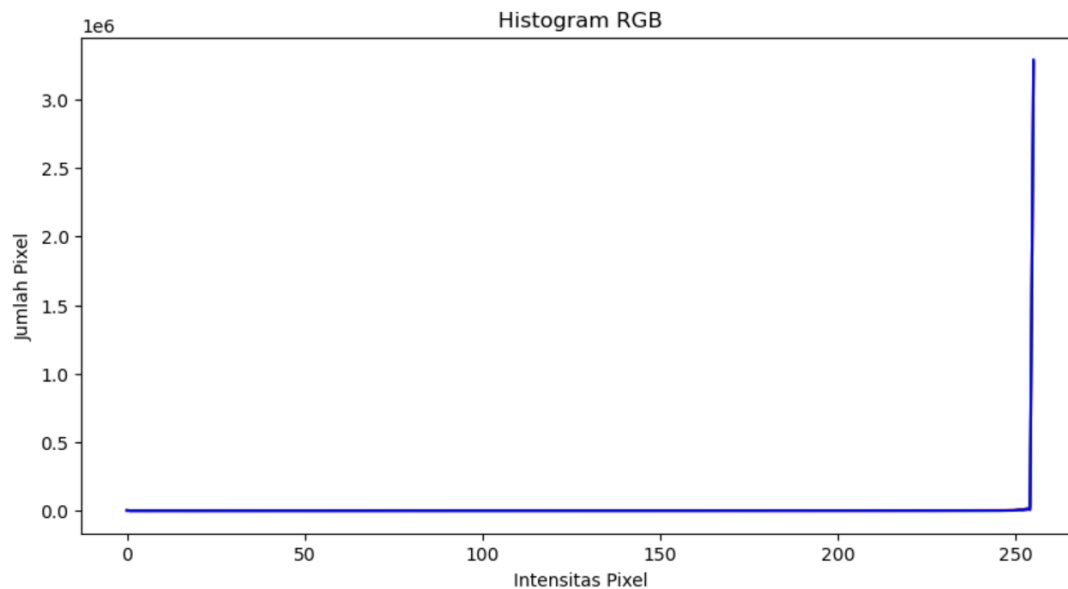
for i, color in enumerate(colors):
    hist = cv2.calcHist([rgb], [i], None, [256], [0,256])
    plt.plot(hist, color=color)

plt.title('Histogram RGB')
plt.xlabel('Intensitas Pixel')
plt.ylabel('Jumlah Pixel')

plt.show()
```

menampilkan kode program untuk pengolahan citra backlight. Program pertama-tama membaca citra asli yang diambil dalam kondisi backlight, kemudian mengkonversinya ke grayscale menggunakan

formula luminance standar melalui fungsi `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`. Selanjutnya diterapkan operasi penjumlahan nilai piksel (brightness enhancement) dan perkalian nilai piksel (contrast enhancement) menggunakan operasi aritmetika NumPy. Fungsi `np.clip()` digunakan untuk memastikan nilai piksel hasil operasi tetap berada dalam rentang 0-255, mencegah terjadinya overflow yang akan mengakibatkan efek color burn berlebihan.



menampilkan histogram citra backlight sebelum dan sesudah dilakukan operasi peningkatan kualitas. Histogram citra asli backlight menunjukkan konsentrasi piksel yang sangat tinggi pada nilai intensitas rendah (sisi kiri histogram), mencerminkan kondisi underexposed pada area subjek yang gelap. Setelah proses peningkatan kecerahan dan kontras diterapkan, distribusi histogram bergeser ke arah kanan dengan sebaran yang lebih luas, menandakan bahwa intensitas piksel pada area gelap berhasil ditingkatkan. Pergeseran histogram ini secara visual menghasilkan citra grayscale yang lebih terang dan memiliki detail subjek yang lebih terlihat jelas dibandingkan citra aslinya.

## **BAB IV**

### **PENUTUP**

Pertama, ekstraksi kanal warna pada citra digital dapat dilakukan secara efektif menggunakan library OpenCV dan NumPy pada Python. Dengan memisahkan komponen R, G, dan B dari citra nama yang ditulis tangan, setiap warna tulisan berhasil diisolasi dan ditampilkan secara individual. Hasil ekstraksi menunjukkan bahwa kanal merah mendominasi area tulisan merah, kanal hijau mendominasi tulisan hijau, dan kanal biru mendominasi tulisan biru, sementara latar belakang putih tetap memiliki intensitas tinggi pada ketiga kanal secara bersamaan.

Kedua, penentuan nilai ambang batas (threshold) yang tepat merupakan kunci keberhasilan proses deteksi warna. Melalui analisis histogram setiap kanal, nilai ambang batas optimal dapat ditemukan pada titik lembah (valley) distribusi histogram yang memisahkan kelompok piksel latar belakang dari piksel tulisan. Penggunaan kombinasi ambang batas antar kanal (seperti RED-BLUE dan RED-GREEN-BLUE) terbukti meningkatkan fleksibilitas dan sensitivitas sistem deteksi, memungkinkan identifikasi piksel dengan karakteristik warna campuran yang lebih kompleks.

Ketiga, histogram citra terbukti menjadi alat analisis yang sangat informatif dalam pengolahan citra digital. Distribusi histogram yang condong ke sisi kanan mengindikasikan dominasi piksel terang (latar belakang putih), sedangkan puncak di sisi kiri merepresentasikan area tulisan yang lebih gelap. Perbandingan histogram sebelum dan sesudah pengolahan memungkinkan evaluasi kuantitatif terhadap efektivitas teknik-teknik yang diterapkan.

Keempat, teknik operasi piksel berupa konversi grayscale, peningkatan kecerahan, dan peningkatan kontras terbukti efektif dalam memperbaiki kualitas citra backlight. Konversi ke grayscale menggunakan formula luminance standar ( $Y = 0.299R + 0.587G + 0.114B$ ) menghasilkan representasi intensitas yang akurat sesuai persepsi mata manusia. Penerapan operasi penambahan dan perkalian nilai piksel yang dikombinasikan dengan fungsi `np.clip()` berhasil meningkatkan visibilitas detail subjek pada area gelap, sekaligus meminimalkan efek color burn pada area yang sudah terang. Secara keseluruhan, project ini membuktikan bahwa teknik-teknik pengolahan citra digital dasar yang dipelajari dalam mata kuliah ini memiliki aplikasi praktis yang signifikan dan dapat diimplementasikan secara efektif menggunakan bahasa pemrograman Python.

**DAFTAR PUSTAKA**

- [1] Gonzalez, R. C., & Woods, R. E. (2022). *Digital Image Processing* (4th ed.). Pearson Education.
- [2] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.
- [3] Hidayatulloh, T., & Nugroho, A. (2021). Implementasi Deteksi Warna pada Citra Digital Menggunakan Metode Thresholding RGB untuk Klasifikasi Objek. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, 8(3), 581–590. <https://doi.org/10.25126/jtiik.2021833374>
- [4] Prasetyo, A., Sari, D. P., & Wibowo, M. (2022). Analisis Histogram Citra Digital untuk Peningkatan Kualitas Gambar dengan Histogram Equalization. *Jurnal Informatika dan Sistem Informasi (JiSi)*, 3(1), 45–56.
- [5] Rahmawati, N., & Kurniawan, B. (2023). Pengolahan Citra Digital untuk Koreksi Pencahayaannya Gambar Backlight Menggunakan Teknik Gamma Correction dan CLAHE. *Jurnal Ilmiah Teknik Elektro Komputer dan Informatika (JITEKI)*, 9(2), 112–124. <https://doi.org/10.26555/jiteki.v9i2.26001>
- [6] Mustafa, R., & Sutrisno, H. (2022). Ekstraksi Fitur Warna Citra Menggunakan Metode Color Channel Separation Berbasis OpenCV. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, 6(4), 1823–1831.
- [7] Purnama, I. K. E., & Hariadi, M. (2021). Segmentasi Citra Berwarna Menggunakan Thresholding Adaptif pada Ruang Warna RGB dan HSV. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 10(3), 234–242. <https://doi.org/10.22146/jnteti.v10i3.2005>
- [8] Wijaya, A. D., & Setiawan, E. (2023). Operasi Piksel pada Pengolahan Citra Digital: Studi Komparatif Metode Brightness Enhancement dan Contrast Stretching. *Jurnal Teknik Informatika dan Sistem Informasi*, 9(1), 78–90.